

The background of the slide is split into two main visual areas. On the left, there is a black and white photograph of chess pieces, with a knight in the foreground being in sharp focus while other pieces in the background are blurred. On the right, there is a dark background featuring a glowing neural network diagram with nodes and connecting lines, some of which have arrows pointing to the right.

Chess Trainer: Marco Teórico de Machine Learning

Arquitectura algorítmica para el análisis y entrenamiento ajedrecístico.

Versión 1.0 | Fecha: 4 de Febrero de 2026

Tagline: De la Estadística Básica a la Intuición Profunda

Visión del Proyecto

Objetivos de Inteligencia Artificial



Predicción de Error

Clasificar jugadas en etiquetas semánticas (`error_label`: blunder, mistake, inaccuracy, good, brilliant).



Reconocimiento de Patrones

Identificación automática de motivos tácticos (pins, forks, discovered attacks).



Análisis Temporal

Detección de `error_streaks` (rachas de errores) y colapsos psicológicos.



Perfilado de Estilo

Segmentación de jugadores (Clusters) basada en métricas de rendimiento y estilo.

Fundamentos: Regresión Lineal

Modelando el valor de una posición.

Teoría

$$y = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n + \epsilon$$

- **Target:** `score\_diff` (diferencia de evaluación en centipawns)
- **Features Key:**
 - `material\_balance`
 - `self\_mobility`
 - `is\_center\_controlled`

Código

Code Card

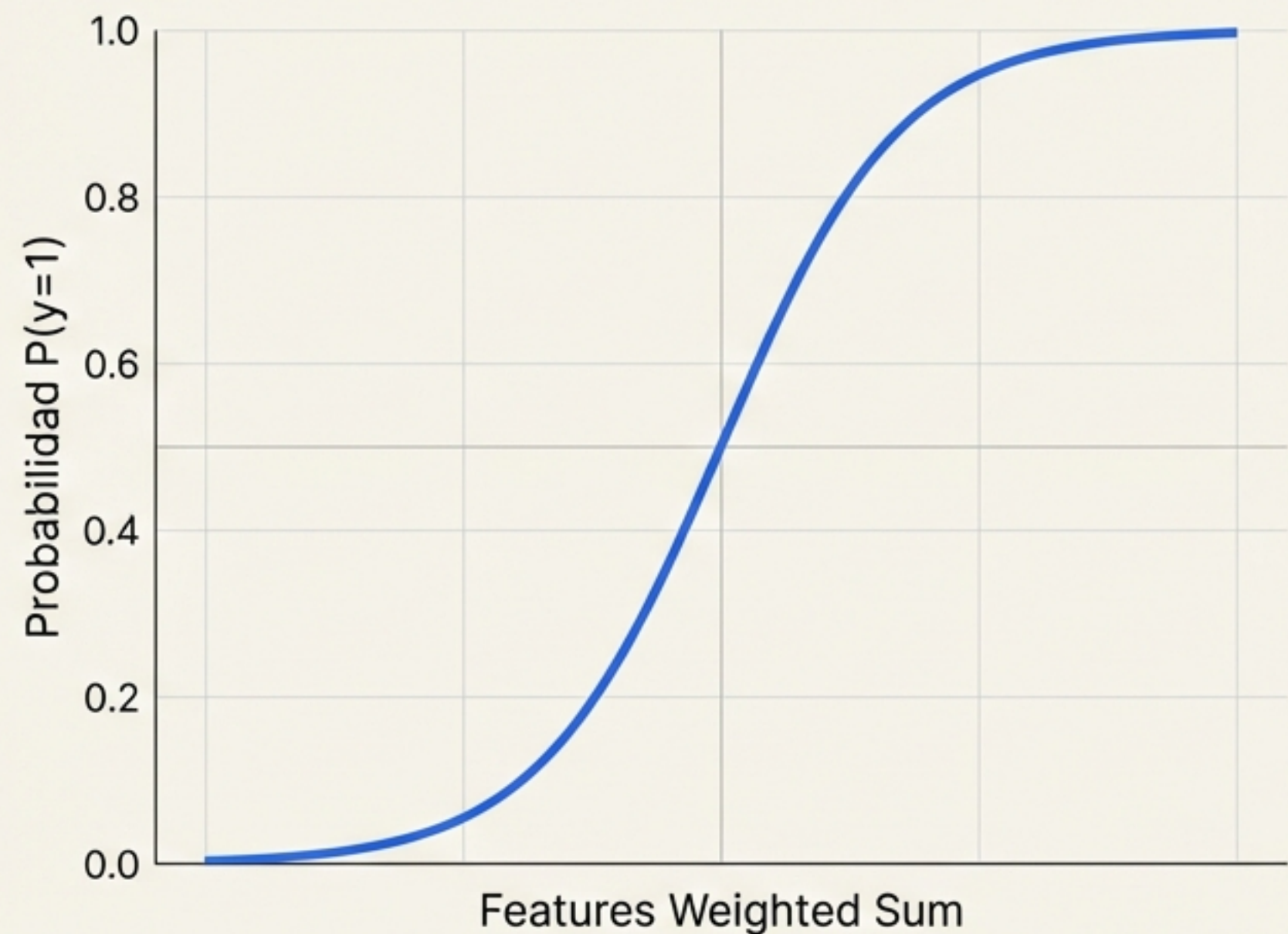
```
model = LinearRegression()
model.fit(X, y)

# Ejemplo de predicción
new_position = {
    'material_balance': 2,
    'branching_factor': 35,
    'self_mobility': 28,
    'is_center_controlled': 1
}
predicted_score = model.predict([list(new_position.values())])[0]
```

Score diferencial predicho: 182.45 centipawns

Regresión Logística

La base de la clasificación de errores (Fase 1).



Métricas de Desempeño

🔗 **Función:** Determinar la probabilidad de que una jugada sea un `blunder`.

KPI Box:

Baseline Fase 1: F1 Macro > 0.70

Blunder Precision:	0.82
Good Precision:	0.88

Prediction Sample

Input: Move Features

Output: `blunder`: 8.45% | `good`: 65.32%

Prediction: Good

Optimización y Selección de Features

Regularización L1 (Lasso) vs L2 (Ridge)

L1 (Lasso)	L2 (Ridge)
• Propósito: Feature Selection • Mecanismo: Fuerza coeficientes a 0 • Uso ideal: Sparse features (aperturas)	• Propósito: Shrinkage (Reducción) • Mecanismo: Reduce magnitud sin eliminar • Uso ideal: Cuando todos los features importan (error_label)

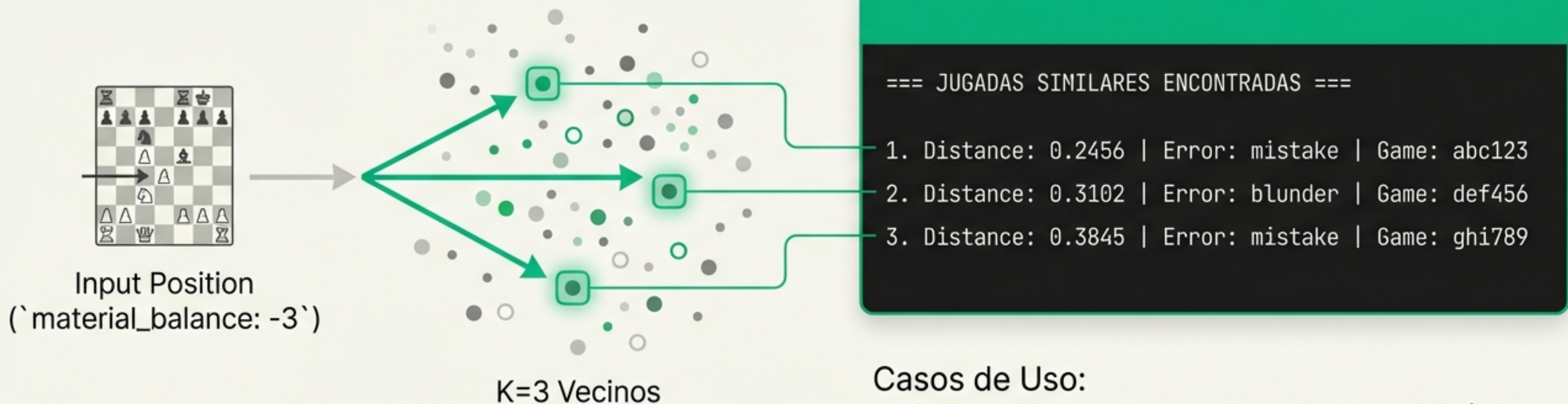
Code Card

```
# Feature Selection con L1
model_l1 = LogisticRegression(penalty='l1', solver='saga')
model_l1.fit(X_train, y_train)

# Resultado
print("Features activos: 8") # vs 14 sin regularización
```


K-Nearest Neighbors (KNN)

El motor de “Déjà Vu” y Sistemas de Recomendación.

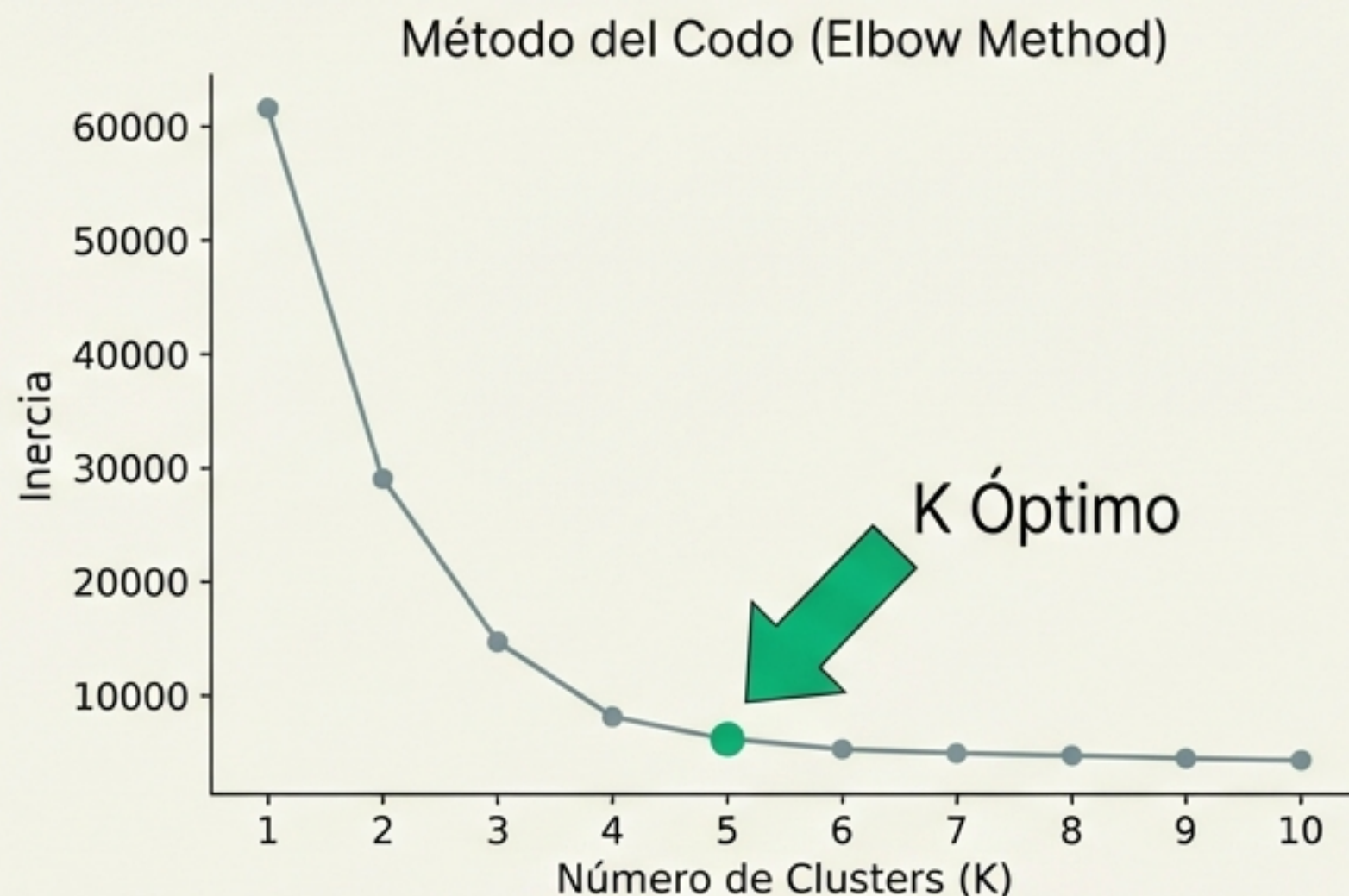


Casos de Uso:

- **Similar Moves:** Encontrar partidas históricas con la misma estructura táctica.
- **Training:** Si un jugador falla, recomendar 5 ejercicios “vecinos” de su historial de errores.

K-Means Clustering

Segmentación de Arquetipos de Jugador.



Cluster 0 (Principiantes):

ELO ~1150 | Blunder rate 8.45%

Cluster 2 (Avanzados):

ELO ~1850 | Blunder rate 2.10%

Cluster 4 (Masters):

ELO ~2450 | Blunder rate 0.30%

Insight: Las recomendaciones de entrenamiento se personalizan según el cluster asignado.

Naive Bayes

Velocidad extrema para detección táctica.

$$P(y|X) = \frac{P(X|y) \cdot P(y)}{P(X)}$$

BernoulliNB

Uso: Features binarios (¿Hay pin? ¿Hay fork?)

Accuracy: 0.7856

Ventaja: Latencia < 5ms en tiempo real

MultinomialNB

Uso: Clasificación de aperturas (frecuencias)

Ejemplo: Predicción "Ruy Lopez"

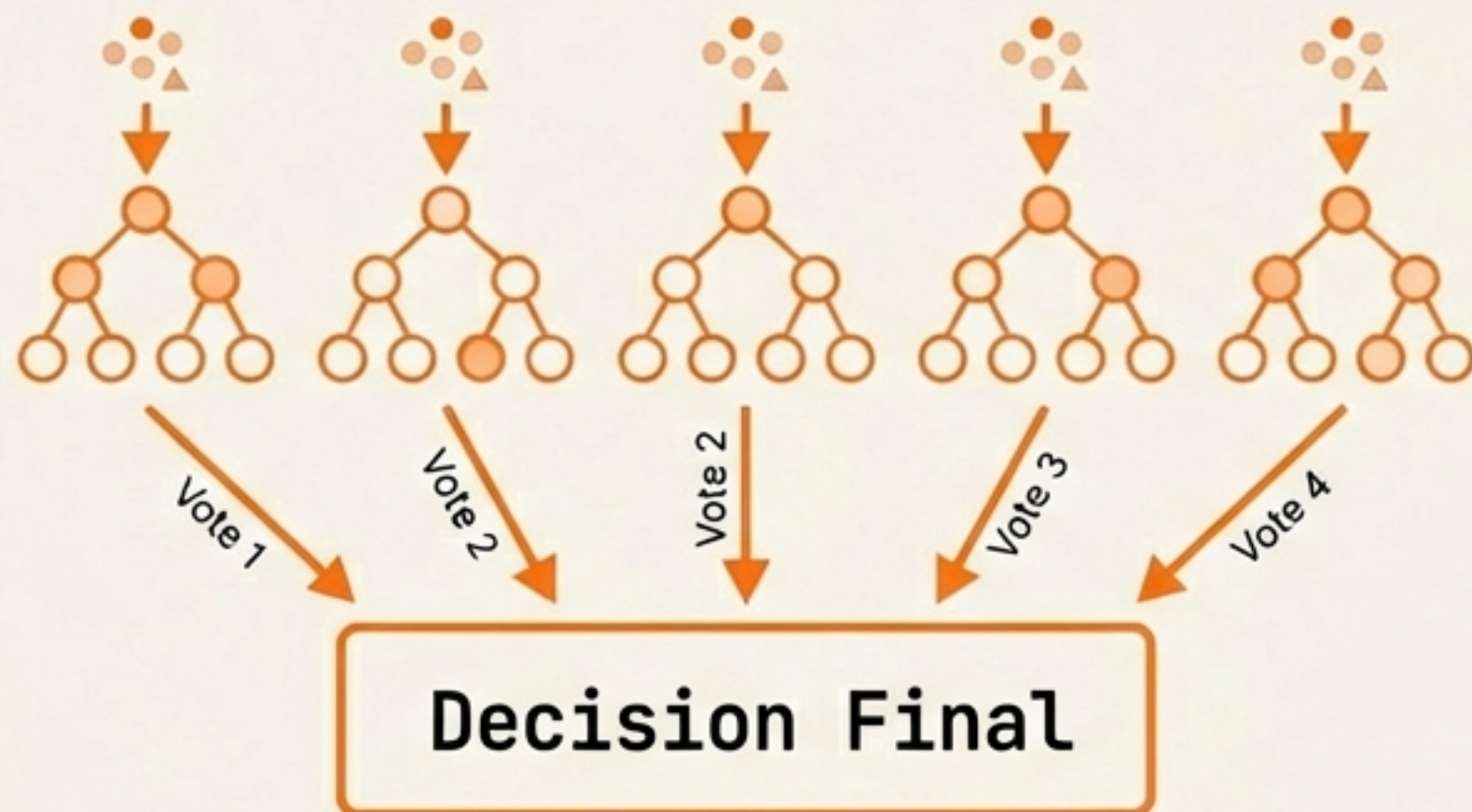
Confianza: 78.45%

“La asunción de independencia permite una velocidad de predicción inigualable.”

Random Forest Ensemble

El clasificador robusto y explicable.

Ensemble Learning



Feature Importance



Validación:

- OOB Score: 0.8456
- Confusión (Good → Blunder): 2.67% (Aprobado < 5%)

Gradient Boosting (XGBoost & CatBoost)

El estándar de producción para máxima precisión.

Champion vs Challenger			
1	Random Forest	F1 Score: 0.8245	Baseline
2	CatBoost	F1 Score: 0.8645	Best for Categorical Data
3	XGBoost	F1 Score: 0.8623	Estándar de Producción

```
Production Configuration

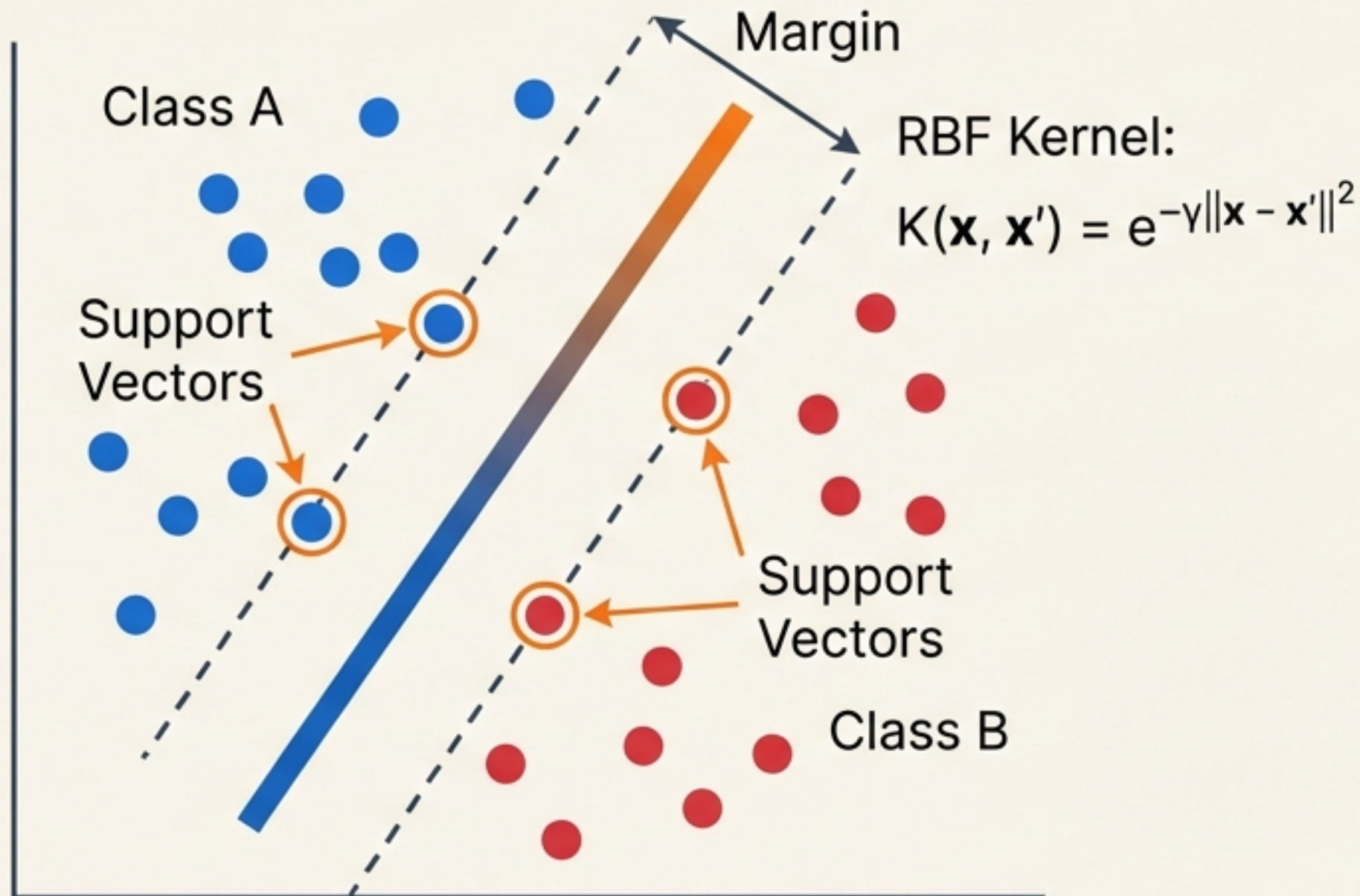
# Configuración de Producción
xgb = XGBClassifier(
    learning_rate=0.1,
    max_depth=6,
    scale_pos_weight=5 # Para desbalanceo
)
```

$$L(\phi) = \sum l(y_i, \hat{y}_i) + \boxed{\sum} \Omega(f_k)$$

Regularización incorporada previene overfitting

Support Vector Machines (SVM)

Cirugía de precisión para fronteras de decisión complejas

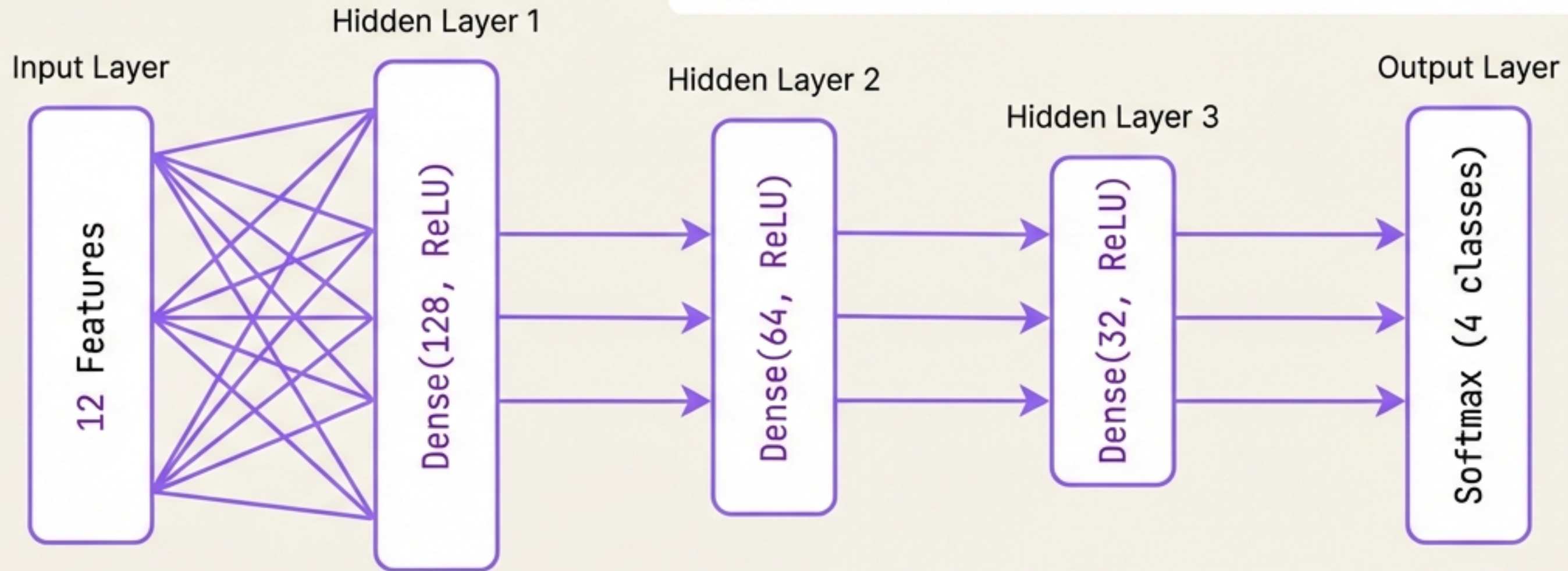


- **Aplicación:** Detección binaria de `blunder` vs `no-blunder`.
- **Accuracy:** 0.9234
- **Trade-off:** Alta precisión matemática vs. lentitud de entrenamiento $O(n^3)$.

Deep Learning: Neural Networks (MLP)

Fase 2 - Capturando la intuición no lineal.

Regularization: BatchNormalization + Dropout(0.3)



Performance: Test F1 Macro **0.8678**

Implementación de Arquitectura MLP

Código de ingeniería para Keras/TensorFlow.

Code Card

```
def create_mlp_model():
    model = keras.Sequential([
        # Capas Ocultas con Regularización
        layers.Dense(128, activation='relu', kernel_regularizer=regularizers.l2(0.01)),
        layers.BatchNormalization(),
        layers.Dropout(0.3),

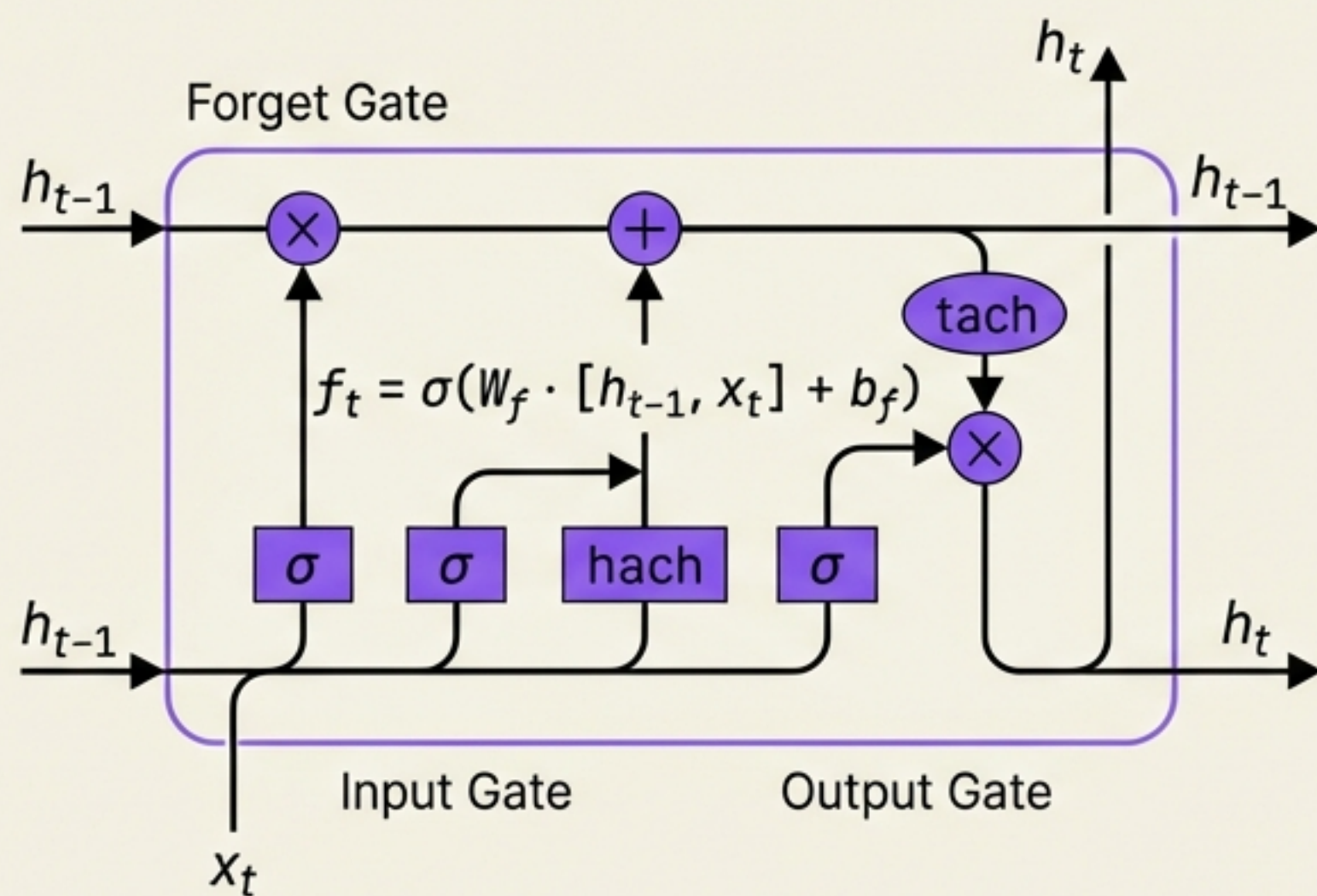
        layers.Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.01)),
        layers.BatchNormalization(),
        layers.Dropout(0.3),

        # Capa de Salida
        layers.Dense(n_classes, activation='softmax')
    ])

    # Optimizador y Callbacks
    optimizer = keras.optimizers.Adam(learning_rate=0.001)
    # Includes EarlyStopping & ReduceLROnPlateau
    return model
```


Recurrent Networks (LSTM/GRU)

Fase 3 - La psicología del tiempo y rachas de errores.



Problema: `Error Streaks`. ¿Cómo afecta un error en la jugada T a la jugada T+1?

Comparativa:

- **LSTM:** F1 0.7956 | Memoria a largo plazo (Slower).
- **GRU:** Accuracy 0.8045 | Alternativa rápida.

Input:

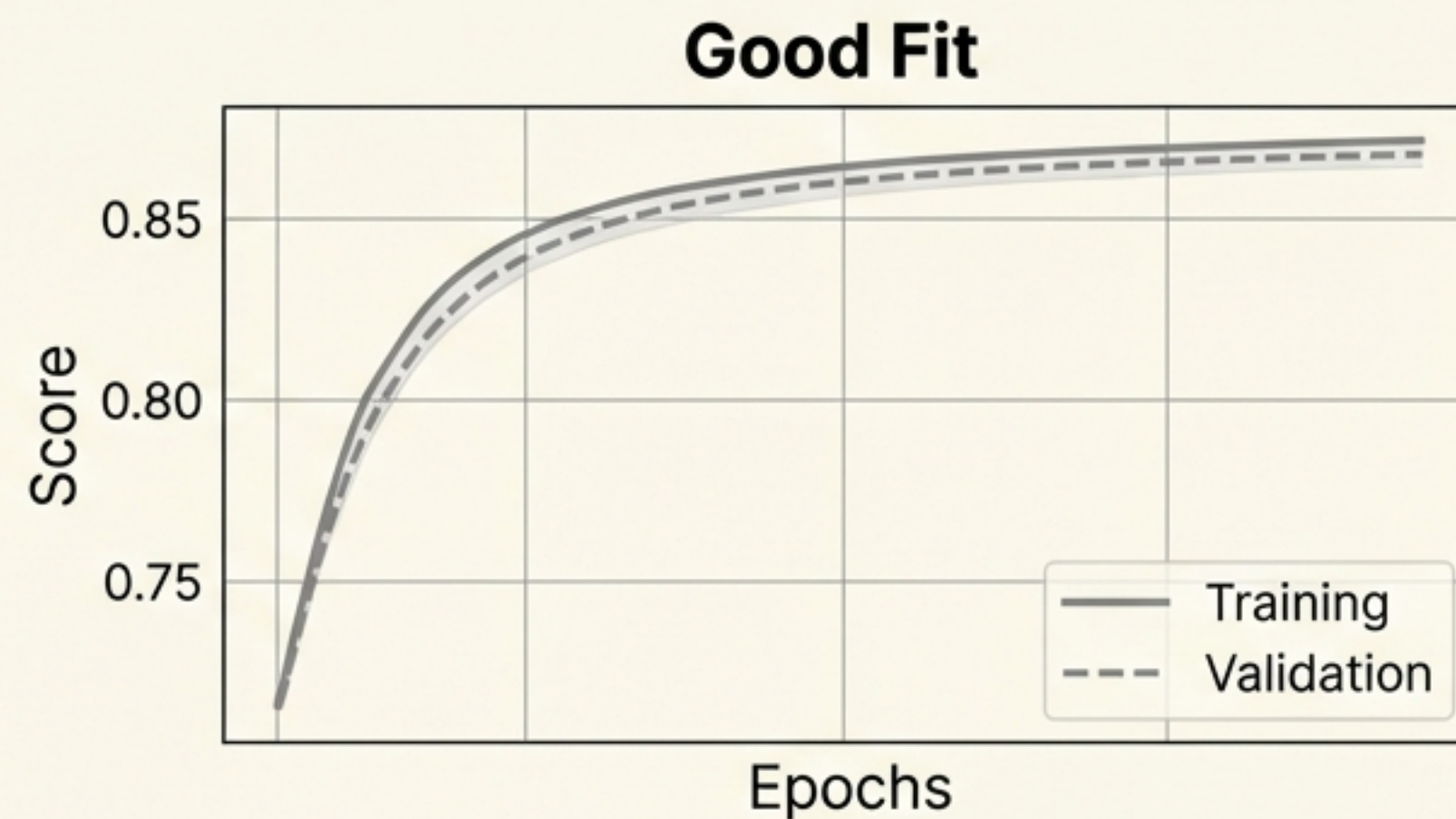
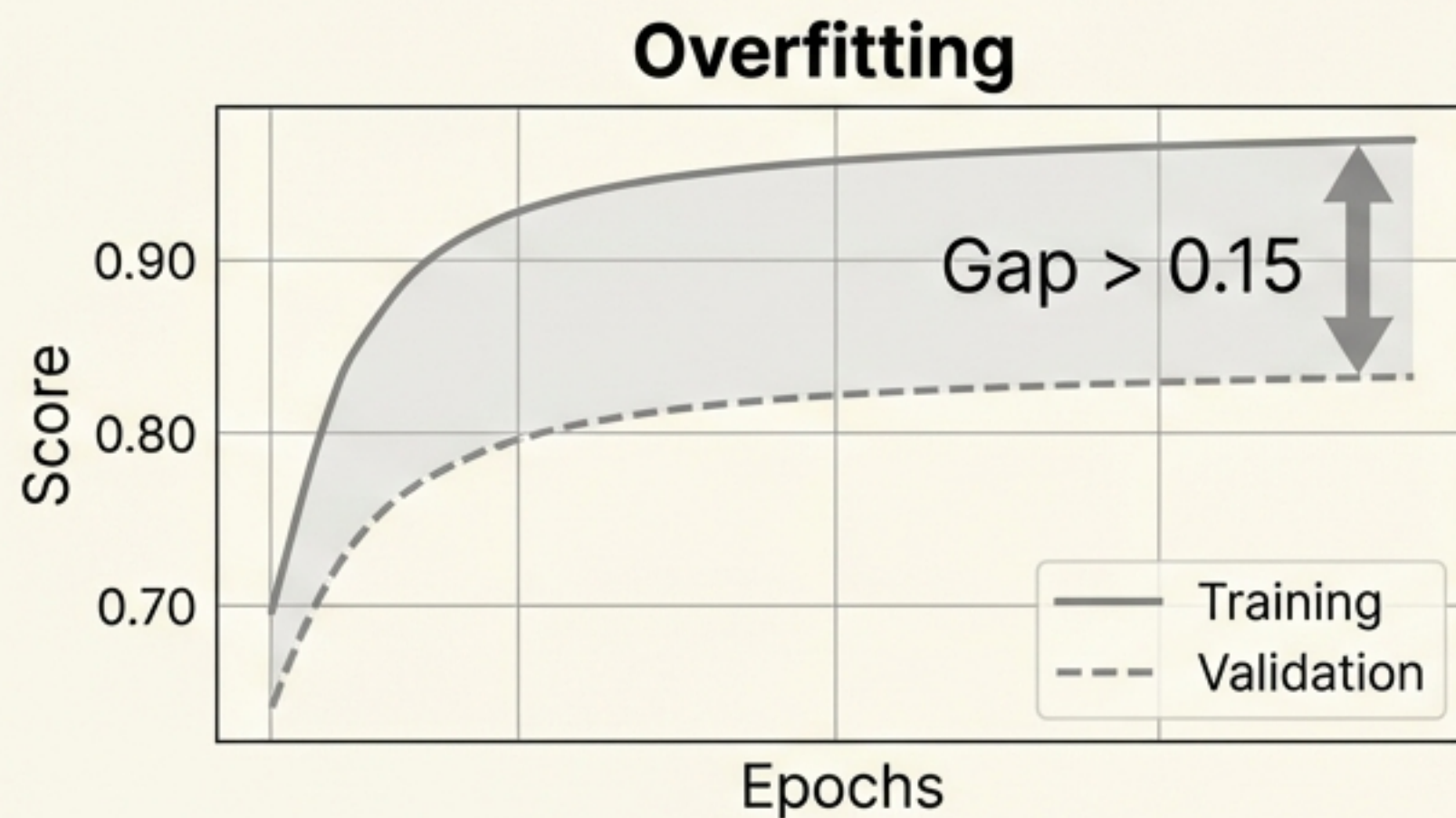
Secuencia de últimas 7 jugadas.

Output:

Riesgo de racha: **! SÍ** (Prob > 0.6).

Gobernanza del Modelo: Overfitting vs. Subajuste

Criterios de Aceptación para Chess Trainer.



Criterios de Aceptación:

- [✓] F1 Macro > 0.70 (Test)
- [✓] Gap (Train - Test) < 0.10
- [✓] Confusión Grave (Good ↔ Blunder) < 5%

Matriz de Selección: Clasificación

Guía de decisión para `error\_label` (Prioridad #1).

Algoritmo	Rol	Performance (F1)	Uso
Logistic Regression	Baseline	< 0.80	Inicio del proyecto
Random Forest	Strong Baseline	~ 0.82	Análisis de Feature Importance
XGBoost	Producción Fase 1	~ 0.87	Estándar actual con L2
MLP (Neural Net)	Upgrade Fase 2	> 0.87	Si XGBoost < 0.85

Matriz de Selección: Tareas Especializadas

Regresión (**score_diff**)

- Algoritmo: XGBoost Regressor
- Target: MAE < 120 cp

Clustering (**player_level**)

- Algoritmo: K-Means (**k=5**)
- Alt: Gaussian Mixture para calidad

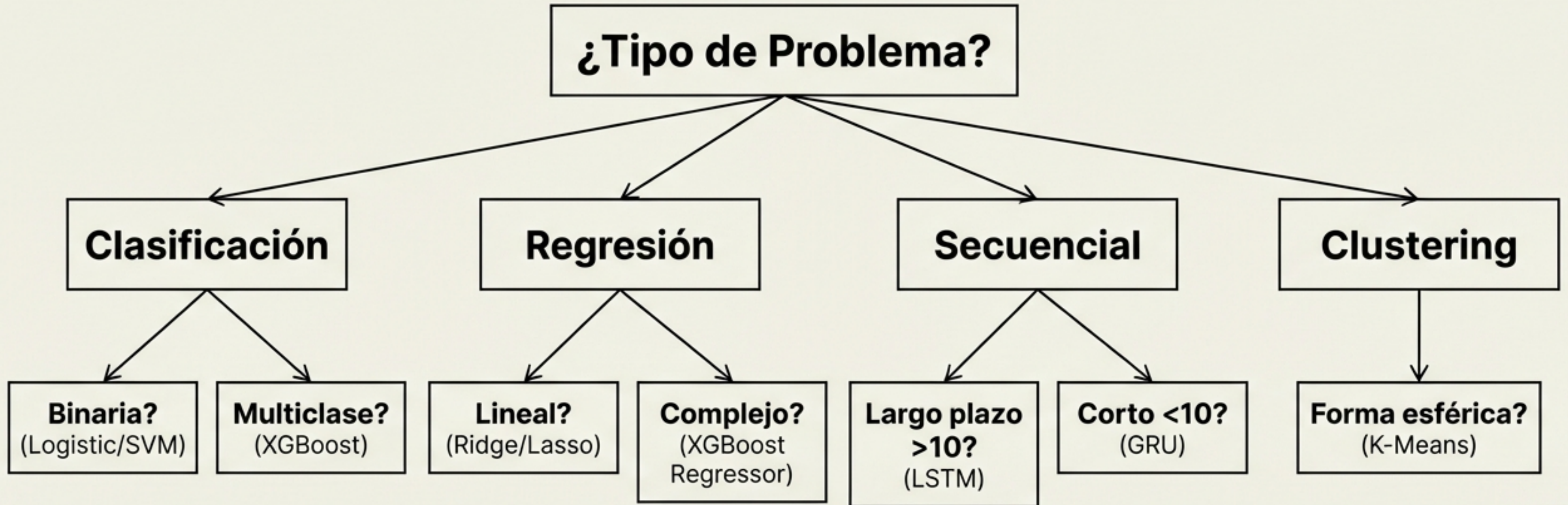
Secuencial (**error_streak**)

- Algoritmo: GRU
- Balance: Accuracy (0.82) / Latencia (~50ms)

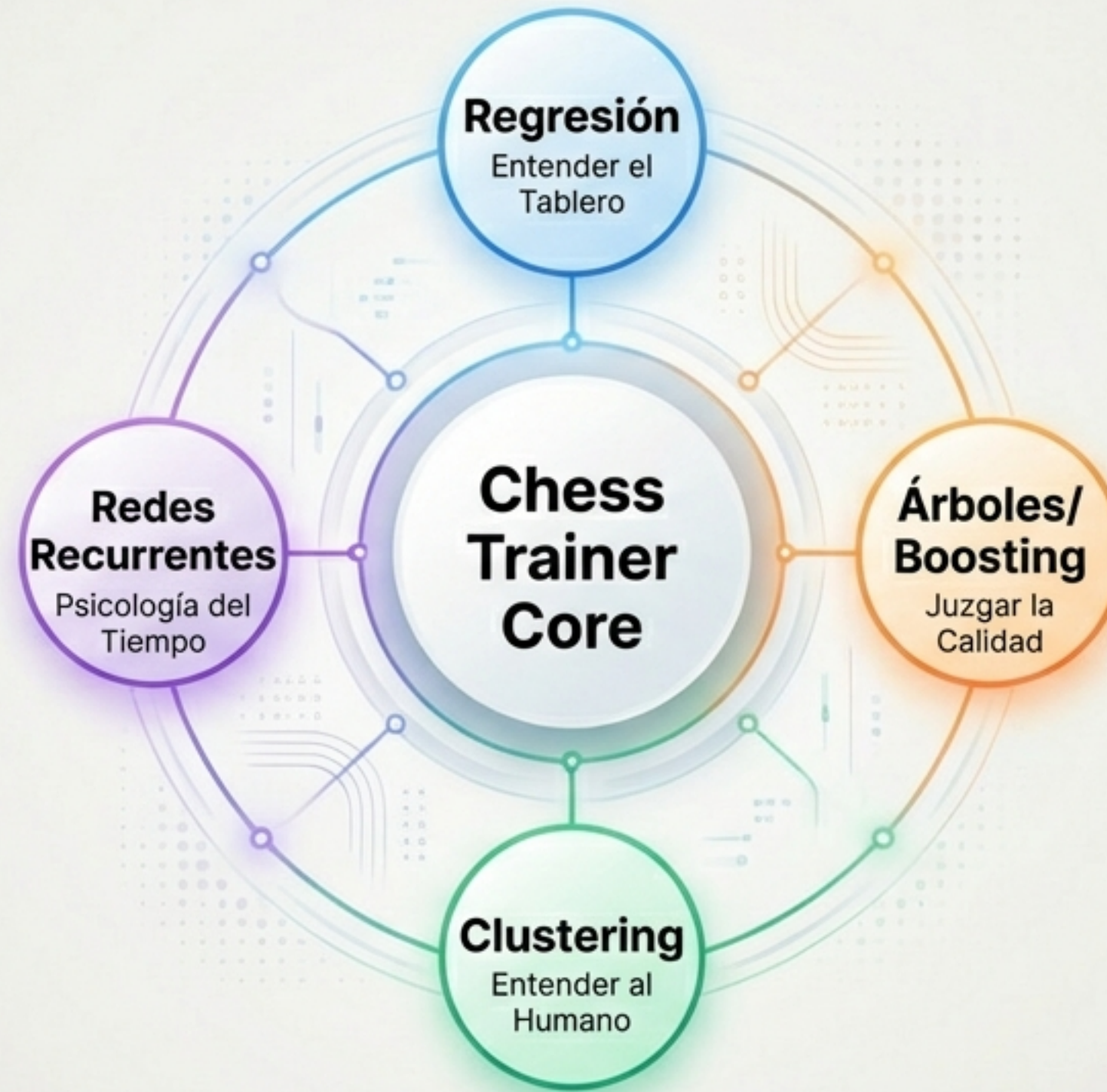
Binario (**blunder_prob**)

- Algoritmo: XGBoost
- Config: scale_pos_weight=**5** (Desbalanceo)

Flujo de Decisión Algorítmica



Resumen del Framework



El sistema actual alcanza un **F1 > 0.86**, superando los benchmarks de métodos tradicionales.

Referencias y Documentación

Documento Fuente:

ML_THEORETICAL_FRAMEWORK.pdf
(v1.0) in JetBrains Mono

Fecha: Febrero 2026



CatBoost



Keras

“La arquitectura de la inteligencia no es magia; es estadística aplicada, estructura y optimización.”